

Federated Learning Under the Lens of Task Arithmetic*

Erik Scolaro

s343553@studenti.polito.it

Marco Roman

s343781@studenti.polito.it

Davide Randino

s346257@studenti.polito.it

Adrien Trahan

s350155@studenti.polito.it

Abstract

Federated learning (FL) enables collaborative model training while preserving data privacy. However, adapting large pre-trained Vision Transformers (ViTs) to heterogeneous, non-IID data remains challenging. In this work, we investigate the efficacy of sensitivity-aware sparse fine-tuning for adapting a DINO-pretrained ViT backbone to CIFAR-100. Specifically, we adopt the least-sensitive parameter selection strategy proposed by the TaLoS framework, which relies on Fisher information to identify and update only the weights with minimal impact on the pre-trained representation. Given the computational constraints of edge devices, we empirically evaluate whether this selection provides a significant advantage in a federated setting, or whether performance gains are primarily driven by the regularization effect of sparsity itself. To address this question, we conduct an ablation study comparing sensitivity-based masking against alternative strategies, including most-sensitive (inverse), magnitude-based, and random masking. Our analysis aims to decouple the benefits of the selection criterion from the general mechanism of sparse updates, and to assess whether the low-sensitivity prior is optimal for preventing catastrophic forgetting and improving robustness to client drift.

1. Introduction

Federated learning (FL) is a machine learning setting where many clients (e.g., mobile devices or entire organizations) collaboratively train a model under the orchestration of a central server (e.g., a service provider), while keeping the training data decentralized [4]. A common use case of this technology is enabling training on images stored locally on users’ personal devices.

To handle complex visual tasks within such distributed

frameworks, modern architectures such as Vision Transformers (ViTs) have become increasingly prominent. These self-supervised feature extractors exhibit strong performance when combined with simple k -Nearest Neighbor (k -NN) classifiers, even without any fine-tuning [1].

To leverage this property, we build a lightweight classifier on top of the backbone using an iCaRL-inspired procedure: rather than learning a parametric head from scratch, we extract exemplar feature vectors and form class prototypes by averaging them into centroids [5]. We keep this prototype-based classifier fixed throughout training, so that federated learning focuses on adapting the ViT feature extractor.

Building on the need for efficient model training, our work explores the use of Task Arithmetic in federated learning environments. Task Arithmetic is a promising approach for identifying task vectors that can be used to edit existing models by injecting or removing such vectors, or to reduce training time by updating only the weights relevant to a specific task.

In the context of image recognition, we evaluate the benefits of applying Task Arithmetic in federated learning. We further investigate unconventional masking strategies and confirm that least-sensitive weights remain the most effective criterion for mask calibration.

2. Related Work

Federated Learning. Federated Learning (FL) enables collaborative training of machine learning models across decentralized edge devices without exchanging raw data. As detailed in the comprehensive survey by [4], FL addresses critical challenges such as data privacy, communication efficiency, and statistical heterogeneity. In the *cross-device* setting, which often involves millions of unreliable mobile devices, communication bandwidth and client availability are primary bottlenecks. To mitigate these issues, standard algorithms such as Federated Averaging (FedAvg) aggregate model updates from a subset of clients. However, the

*Code available at: <https://github.com/erikscolaro/FL-task-arithmetic.git>

field continues to evolve to address open problems such as robustness, personalization, and efficient training of large-scale models in resource-constrained environments [4].

Self-Supervised Vision Transformers. While Convolutional Neural Networks (CNNs) have long dominated computer vision, Vision Transformers (ViTs) have recently emerged as a powerful alternative. Caron et al. [1] studied ViTs trained via self-supervised learning and introduced the DINO (Self-Distillation with No Labels) framework. Their work showed that self-supervised ViTs learn highly structured feature representations, achieving competitive performance with simple k -Nearest Neighbor (k -NN) classifiers even without fine-tuning. This indicates that the learned feature space is inherently organized by semantic classes, making it well suited for prototype-based and non-parametric classification methods [1].

iCaRL is a class-incremental learning strategy that combines a deep representation with a nearest-mean-of-exemplars classifier. Instead of relying solely on the network’s output layer, iCaRL maintains a small exemplar set per class and classifies new samples based on their proximity to class centroids in feature space [5]. By rehearsal on exemplars (and distillation in the original formulation), it mitigates catastrophic forgetting when new classes are introduced sequentially.

Task Arithmetic and Model Editing. Efficiently adapting large pre-trained models to specific tasks is a growing area of research. Task Arithmetic [2] treats “task vectors”, the difference between a fine-tuned model and its pre-trained initialization, as composable units that can be added or subtracted to edit model behavior. Building on this, [3] proposed TaLoS (Task-Localized Sparse Fine-Tuning), a method to construct sparse task vectors that minimize interference when composed. They observe that pre-trained models contain parameters with consistently low gradient sensitivity. By restricting updates to these parameters, they achieve weight disentanglement without expensive linearization. This methodology is particularly relevant for federated learning, where efficient, modular updates can reduce communication costs.

3. Methodology

In this section, we detail the experimental framework adopted to evaluate the efficacy of sparse fine-tuning techniques in a Federated Learning (FL) setting. We first describe the dataset and the heterogeneity simulation, followed by the model architecture, the centralized training baseline, and the federated optimization protocol. Finally, we formally introduce the Task-Localized Sparse Fine-Tuning (TaLoS) approach and the specific masking strategies analyzed in this work.

3.1. Building the dataset

We conduct our experiments on the CIFAR-100 dataset distributed among $K = 100$ clients. To simulate realistic Federated Learning scenarios, we adopt a partitioning strategy based on label availability, where the degree of statistical heterogeneity is controlled by the hyperparameter N_c (number of classes per client).

Each client is restricted to hold samples drawn from exactly N_c distinct classes. In our experimental analysis, we vary $N_c \in \{1, 5, 10, 50, 100\}$ to cover the full spectrum of distribution shifts:

- **Pathological Non-IID** ($N_c = 1$): Represents the most extreme case of statistical heterogeneity (Label Skew), where each client possesses data belonging to a single class.
- **Intermediate Heterogeneity** ($N_c \in \{5, 10, 50\}$): Scenarios with partial access to the global label space, introducing varying levels of client drift.
- **IID Setting** ($N_c = 100$): Since the dataset contains 100 classes, setting $N_c = 100$ implies that each client samples from the entire label space. This configuration is equivalent to a standard IID uniform distribution, serving as our baseline for ideal data homogeneity.

3.2. Model Architecture and Classifier Initialization

As the backbone for feature extraction, we utilize a Vision Transformer (ViT) architecture, specifically the `dino_vits16` variant, pre-trained using the DINO framework [1]. DINO features have been shown to naturally cluster by class, making them highly suitable for distance-based classification tasks.

Following the methodology described in the iCaRL paper [5], we adopt a Nearest Class Mean (NCM) approach to construct our classification head. The construction of the architecture follows a structured three-step pipeline before any federated or centralized training occurs:

1. **Feature Extraction:** We pass the entire CIFAR-100 training split through the DINO ViT-S/16 backbone to extract high-dimensional feature representations.
2. **Centroid Calculation:** Leveraging the ground truth labels, we compute the mean feature vector (centroid) for each of the 100 classes in the dataset.
3. **Linear Layer Conversion:** These class centroids are then used to initialize the weights of a standard linear layer, where each row of the weight matrix corresponds to a class prototype.

The final architecture shared by all the following experiments is thus composed of two main modules:

- **Backbone:** The DINO ViT-S/16 acts as the core trainable component. During the federated phase, this backbone is fine-tuned (either fully or via sparse updates) to align its representations with the predefined class prototypes.

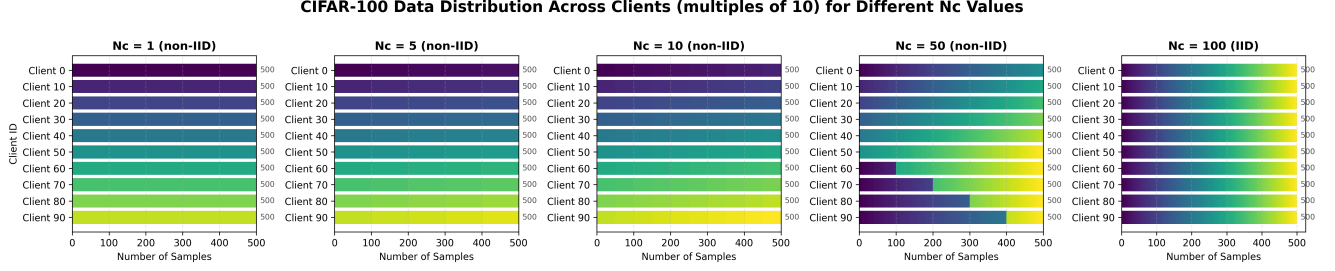


Figure 1. Class distribution across 100 clients for different values of $N_c \in \{1, 5, 10, 50, 100\}$. The figure illustrates the impact of the number of local classes on each client’s dataset composition, highlighting the transition toward a uniform (IID) distribution as N_c increases.

- **Fixed Classifier:** The NCM-initialized linear layer is kept **frozen** throughout the entire project. By fixing the decision boundaries *a priori*, we enforce that the learning process focuses solely on adapting the feature extractor to the data distribution, mitigating potential interference in the classification head during global aggregation.

Upon completing the initialization, the fixed prototype-based classifier achieved a baseline top-1 accuracy of 60.41% on the CIFAR-100 test set.

3.3. Centralized Baseline

To establish an upper bound on performance and validate our architectural choices, we first conducted a centralized training procedure. The model was trained for a maximum of 30 epochs on the full CIFAR-100 training set using the SGDM optimizer. To prevent overfitting and optimize training time, we implemented an early stopping strategy with a patience of 5 epochs, monitoring the validation loss. Through a hyperparameter search, we identified the optimal configuration which serves as the “gold standard” reference for all subsequent federated experiments.

3.4. Federated Optimization Protocol

We employ the standard *Federated Averaging* (FedAvg) algorithm as the aggregation strategy. In each communication round t , the central server selects a fraction $C = 0.1$ of the total $K = 100$ clients. Selected clients receive the current global model weights w_t and perform J local training epochs using SGDM.

To ensure a fair comparison across different local workloads, the total number of communication rounds T is modulated inversely to the number of local epochs J . This protocol maintains a constant global computational budget (i.e., the total number of local SGD updates) across all experiments. Specifically, we evaluate the following hyperparameter pairs: $(J = 4, T = 60)$, $(J = 8, T = 30)$, and $(J = 16, T = 15)$. After the local training phase, the updated weights are sent back to the server and aggregated via weighted averaging to produce the new global model w_{t+1} .

3.5. Federated Task Arithmetic with Sparse Fine-Tuning

To perform efficient model editing and enable Task Arithmetic [2] in a distributed environment, we partially implement the Task-Localized Sparse Fine-Tuning (TaLoS) framework [3]. The core objective is to ensure that model updates generated by different clients are confined to a specific subspace of parameters.

Specifically, this subspace is composed of the **least-sensitive parameters** of the model: those that have minimal impact on the pre-trained features. Intuitively, these are the weights located in “flat” regions of the loss landscape, where small perturbations do not significantly alter the output. By restricting updates to these parameters via a binary mask $M \in \{0, 1\}^d$, we allow the model to adapt to new tasks by modifying only its “flexible” components, while keeping the critical, high-sensitivity backbone frozen to preserve the accumulated pre-trained knowledge.

The implementation of this framework is divided into two phases: the server-side mask calibration and the client-side sparse optimization.

3.5.1. Server-Side Mask Calibration

To generate the binary mask M , inspired by the original TaLoS implementation, we adopt an iterative progressive sparsity strategy.

This approach gradually increases the sparsity level over R_{cal} calibration rounds. This strategy is essential to account for the interdependence among parameters, as masking a subset of weights inevitably alters the sensitivity of the remaining ones. Concretely, rather than immediately pruning weights to zero (which would disrupt gradient flow and lead to layer collapse) we assign a small value (e.g., 0.01) to the masked parameters at each calibration round.

Let S_{target} be the final desired sparsity (e.g., 0.9) and let $\rho_{final} = 1 - S_{target}$ be the fraction of parameters to retain. We define a per-round retention schedule such that at each calibration round $r \in \{1, \dots, R_{cal}\}$, the fraction of active parameters ρ_r is given by the geometric decay:

$$\rho_r = (\rho_{final})^{\frac{r}{R_{cal}}} \quad (1)$$

In each round r , the calibration proceeds as follows:

1. **Sensitivity Computation:** We re-compute the sensitivity scores for the currently active parameters. To approximate the diagonal of the Fisher Information Matrix, we aggregate the squared gradients computed on *individual samples* (x, y) rather than on averaged batches:

$$s_i = \sum_{(x,y) \in \mathcal{D}_{cal}} \left(\frac{\partial \ell(x, y; \theta)}{\partial \theta_i} \right)^2 \quad (2)$$

Note that frozen parameters from previous rounds are excluded from this calculation.

2. **Thresholding and Freezing:** We determine a dynamic threshold τ_r corresponding to the $(1 - \rho_r)$ -th percentile of the current scores. Parameters with the highest sensitivity $s_i \geq \tau_r$ (in the Least-Sensitive strategy) are permanently added to the frozen set ($M_i \leftarrow 0$).
3. **Iterative Refinement and Soft-Masking:** Parameters identified for masking are assigned a small constant value (e.g., 0.01) for the reasons described above. The process repeats.

3.5.2. Client-Side Sparse Optimization

At training time, the server broadcasts both the mask and the initial pre-trained parameters θ_0 to the selected clients. The primary objective is to adapt the global model to local heterogeneous data distributions. Unlike standard federated learning, only the low-sensitivity parameters are trained.

To mathematically enforce this constraint, we employ a custom Sparse Stochastic Gradient Descent (SparseSGDM) optimizer. Beyond masking the gradients, SparseSGDM applies an element-wise multiplication between the computed gradient and the binary mask. This masking is also extended to the momentum buffer at each iteration. This dual application ensures that the velocity vector for any inactive parameter is strictly reset to zero, preventing history-based drift caused by accumulated momentum.

This optimization strategy guarantees that the parameters identified as critical (the “frozen backbone”) remain clamped to their initialization values throughout the entire training process.

3.6. Extension: Alternative Masking Strategies

Standard Task Arithmetic uses Fisher Information to identify specific parameters for tuning, typically freezing the most sensitive ones to preserve pre-trained knowledge. However, we aim to verify whether this specific selection is strictly necessary or if the performance benefits stem simply from the sparsity itself. A key research question is whether the “low-sensitivity” prior is indeed optimal, or if alternative strategies - including the exact opposite approach of tuning the most critical parameters - can yield comparable results.

To decouple the benefits of the selection criterion from the general regularization effect of sparse updates, we compare the standard sensitivity-based approach against a comprehensive set of baselines. We investigate the following strategies to construct the binary mask M , which is then enforced by our custom SparseSGDM optimizer to ensure strict adherence to the selected subset.

Most-Sensitive Masking (Inverse Strategy). We invert the standard protocol by exclusively updating the parameters with the highest Fisher Information scores. Mathematically, given the diagonal of the Fisher Information Matrix F , we construct the mask M such that $M_i = 1$ if F_{ii} falls within the top- k percentile. This strategy serves as a critical stress test for the frozen backbone hypothesis: we aim to verify whether the most informative parameters represent unalterable foundational knowledge, which yields catastrophic forgetting if modified, or whether they are the most effective levers for adapting the model to new domains.

Highest-Magnitude Masking. We restrict updates to the weights with the smallest absolute values ($|w| \approx 0$). Unlike Fisher-based methods that require a calibration phase with forward and backward passes to estimate sensitivity, this approach relies purely on a static analysis of the pre-trained weights θ_0 . It tests the hypothesis derived from pruning literature that “weak” connections contribute minimally to the pre-trained representation and can be treated as free capacity. This offers a computationally inexpensive alternative to Hessian-based calculations, allowing for aggressive fine-tuning without disrupting the core features encoded in larger weights.

Lowest-Magnitude Masking Conversely, we restrict updates to the weights with the largest absolute values ($|\theta_{0,i}| \in \text{top-}k$). Here, we evaluate the competing hypothesis that effective adaptation requires modifying the strongest connections, which are the primary drivers of feature activation in the pre-trained Vision Transformer. This strategy verifies whether modifying the dominant features is necessary to bridge the gap between pre-training and target distributions, despite the significantly higher risk of feature distortion and forgetting.

Random Masking (Control Baseline). We generate a binary mask M uniformly at random, disregarding any weight magnitude or gradient sensitivity information. This acts as our fundamental null hypothesis to isolate the source of improvement. Since our SparseSGDM ensures that masked parameters ($M_i = 0$) have both their gradients and momentum buffer strictly zeroed out, any performance gain here

would imply that the benefits are not attributable to the semantic selection of parameters, but are merely artifacts of the sparsity acting as a regularizer against communication noise in the federated setting.

4. Experiments

In this section, we evaluate our proposed Federated Task Arithmetic approach in a federated setting. Given the computational constraints of edge devices, we focus on isolating the effect of the masking selection rule rather than sparsity alone. To this end, we present an ablation-style comparison of sensitivity-based masking against alternative strategies, including Most-Sensitive (inverse), magnitude-based, and random masking.

4.1. Experimental Setup

Baselines. To select an appropriate learning-rate scheduler and its hyperparameters, we trained the centralized model under four configurations: using CosineAnnealing or no scheduler, and setting the learning rate to either 10^{-4} or 10^{-5} . Preliminary experiments indicated that a learning rate of 10^{-3} did not yield satisfactory performance, and we therefore omit it from subsequent experiments. As shown in Figure 3, the configurations using a learning rate of 10^{-4} achieve the highest accuracy, with the no-scheduler setting performing best at approximately 87.04% accuracy on the test set. The same learning rate is used in all the following experiments.

FL Baselines. We establish strong baselines using Federated Averaging (FedAvg) [4] with varying degrees of parallelism and client participation. Specifically, we evaluate configurations with total client counts $N_c \in \{1, 5, 10, 50, 100\}$ and local steps $J \in \{4, 8, 16\}$.

The CIFAR-100 dataset is split among 100 clients, but for each training round only 10 are selected to receive a copy of the central model for fine-tuning ($C = 0.1$).

The number of training rounds is proportional to J to ensure fair comparisons between models by maintaining constant total computational power across client configurations.

Results. Because of our limited computing power, we performed a single experimental run for each combination of J and N_c .

Nevertheless, Figures 2 confirm the intuition that the maximum accuracy a federated model can achieve is directly proportional to the diversity of data (number of classes) the clients receive. The extreme case of $N_c = 1$ was not able to learn in all settings.

Figure 2 demonstrates that a higher number of local steps (J) initially accelerates accuracy improvement. However, we achieved optimal performance with $J \in \{4, 8\}$, which

reached similar top accuracies. This suggests that allowing client models to train locally for too long ($J = 16$) may lead to overfitting, resulting in a degraded aggregated model.

All models begin at approximately 60% accuracy, as the “vanilla” DINO backbone is evaluated on the same CIFAR-100 partition prior to the fine-tuning process.

4.2. Task Arithmetic

After identifying the optimal local step parameter, we evaluated the performance of Task Arithmetic in a Federated Learning environment. We fixed $N_c = 10$ and $J = 8$, while varying the number of calibration rounds $r \in \{1, 2, 4, 8\}$ and the mask sparsity $s_m \in \{50\%, 70\%, 90\%\}$.

Results. Due to computational constraints, we ran a single trial for each combination of r and s_m ; consequently, we restrict our analysis to an ablation-style comparison of these settings.

As shown in Figure 4 and 5, all configurations match or exceed the corresponding baseline accuracy. The only exception is $r = 8$ at $s_m = 50\%$, which underperforms the baseline. At $s_m = 70\%$, varying the number of calibration rounds has a negligible effect. At higher sparsity ($s_m = 90\%$), fewer calibration rounds tend to yield slightly higher accuracy, whereas at lower sparsity ($s_m = 50\%$) we observe the opposite trend.

Overall, these results suggest that, to achieve better performance, we should use higher sparsity rates and a suitable number of calibration rounds.

4.3. Experimental mask calibration rules

To experiment with new mask calibration rules, we maintained the same strategy and hyperparameters used in the normal Task Arithmetic, $r = 30$ $J = 8$.

This time we did each experiment twice for all other strategies: Most-sensitive, Highest-Magnitude, Lowest-Magnitude, Random.

Results.

1. **Most-Sensitive (inverse) parameters (Figure 6).** As shown in Figure 6, the Most-Sensitive masking strategy yields notably unstable training. This behavior is expected when updates are restricted to a highly irregular subspace induced by selecting parameters with large gradient magnitudes. As we decrease the mask sparsity, performance improves as the model is allowed to update a larger set of parameters with lower gradient magnitudes, leading to smoother and more stable updates.
2. **Highest magnitude parameters (Figure 6).** Freezing the strongest connections in our transformer made us fine-tune the weakest ones. In our experiments, we encountered sudden drops in accuracy and the highest variance among these extension experiments. This may

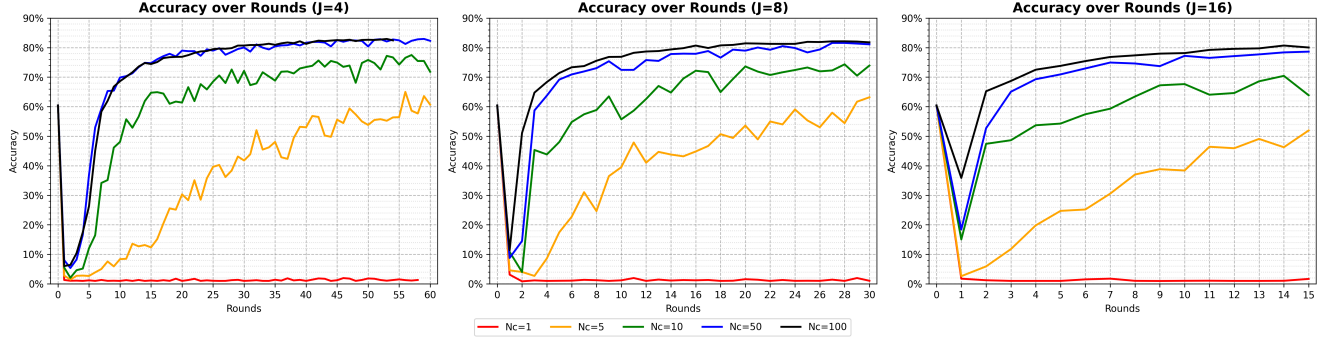


Figure 2. FedAvg baseline: accuracy over different N_c and J

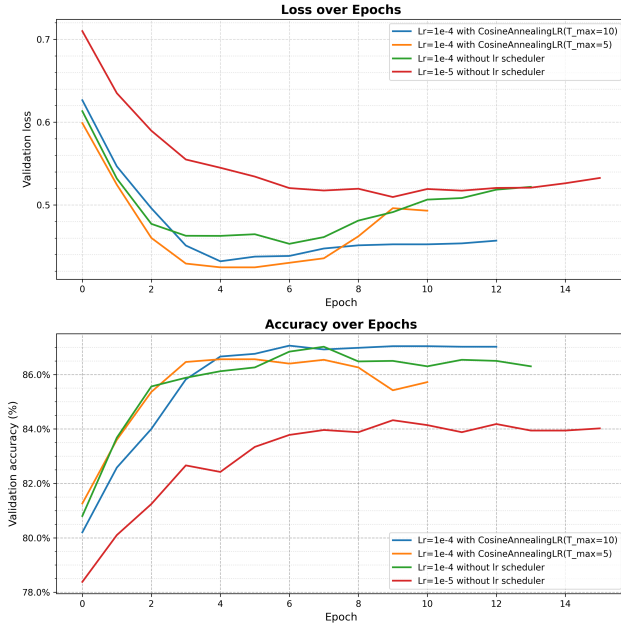


Figure 3. Validation loss and accuracy metrics across different learning rate strategies for the centralized baseline.

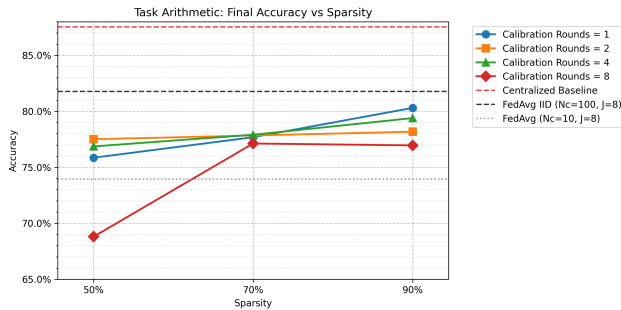


Figure 4. Impact of sparsity levels on model accuracy across various calibration settings.

our weights based on sensitivity, some high-sensitivity weights cause the spikes in accuracy we observe.

3. **Lowest magnitude parameters (Figure 6).** By freezing the lowest-magnitude connections, we instead fine-tune only the strongest weights. This yields a stable training trajectory with low variance and no abrupt spikes. Overall, the results suggest that magnitude is not a reliable proxy for sensitivity: the set of high-magnitude weights does not necessarily coincide with the most sensitive parameters.
4. **Random parameters (Figure 6).** Random masking yields performance comparable to both magnitude-based variants, further suggesting that weight magnitude alone is not an informative criterion for task arithmetic.

Overall, the standard least-sensitive calibration performs best, reaching 80.31% accuracy. In contrast, magnitude-based and random masking achieve around 77.5%. These results further reinforce least-sensitive masking as an effective strategy, while Most-Sensitive masking underperforms and introduces pronounced training instability.

5. Conclusion

In this work, we examined Federated Learning through the lens of Task Arithmetic, specifically investigating the efficacy of sensitivity-aware sparse fine-tuning for adapting a DINO-pretrained ViT backbone. Rather than applying model merging directly, we adopted the core philosophy of the TaLoS framework (the least-sensitive parameter selection strategy) and compared it against alternative masking methodologies (Random, Magnitude-based, and Most-Sensitive). The primary objective was to decouple the benefits of the selection criterion from the general mechanism of sparse updates, determining whether the specific prior based on Fisher Information offers a tangible advantage over stochastic or magnitude-based sparsity in a distributed setting.

Our experimental results delineate a clear performance hierarchy where the specific criterion for parameter se-

be caused by the fact that, since we are not splitting

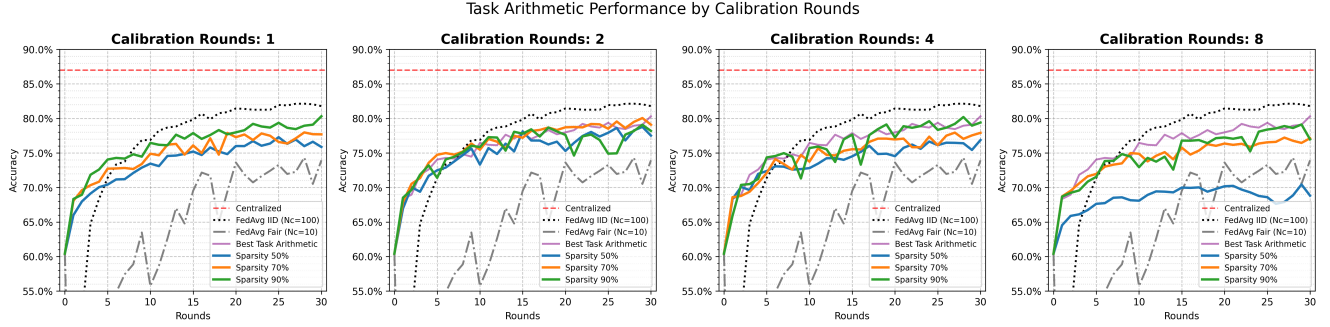


Figure 5. Comparative accuracy curves illustrating the effect of increasing calibration rounds on model performance. The data shows how varying sparsity affects the learning trajectory compared to FedAvg IID and FedAvg Fair baselines

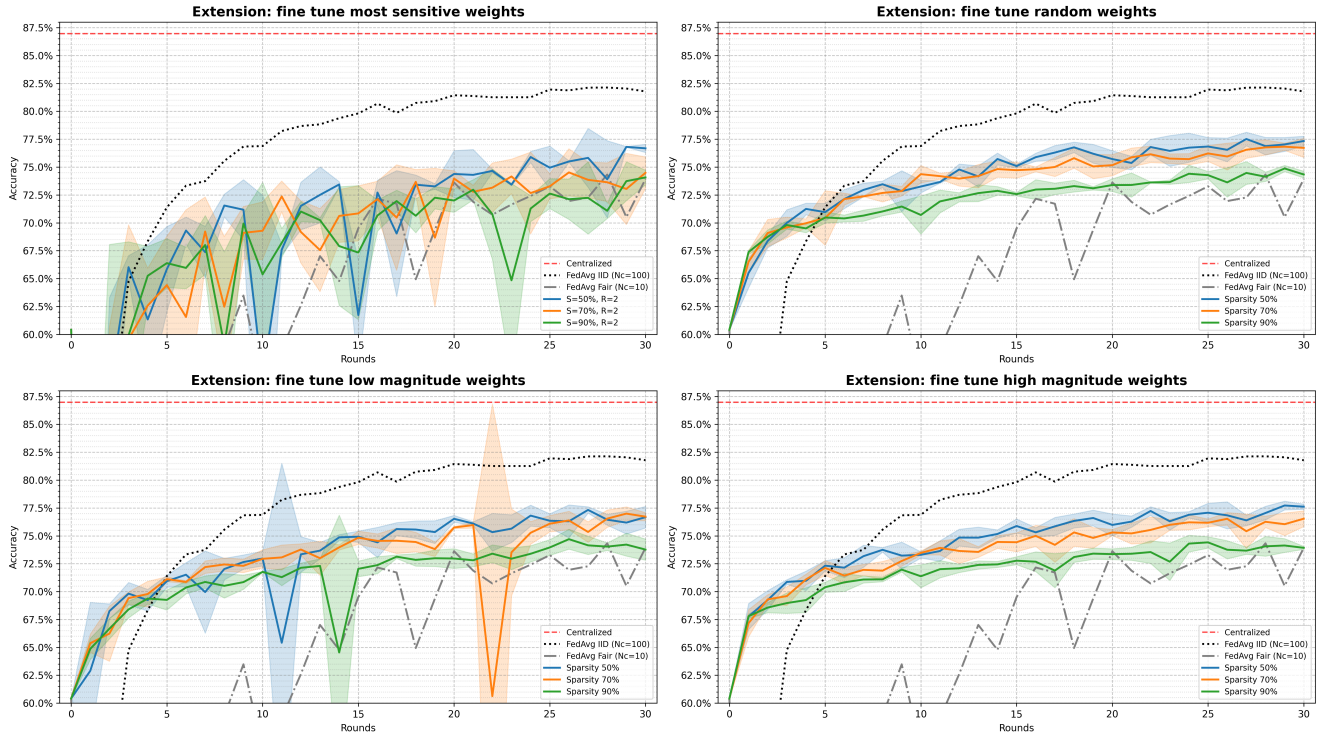


Figure 6. Impact of weight selection strategies on test accuracy: evaluating fine-tuning for sensitive, random, and magnitude-based weight subsets across multiple sparsity levels.

lection dictates both stability and accuracy. The TaLoS-inspired strategy, which updates only the least-sensitive parameters, established itself as the superior approach, achieving an accuracy of approximately 80%, resulting in a margin of roughly 2.5% over the best alternative masking methods. This dominance underscores the critical importance of a “surgical” selection process; by strictly preserving the backbone’s prior knowledge and avoiding updates to weights with high Fisher Information, Task Arithmetic ensures not only peak performance but also robust training stability in the federated setting.

In contrast, while alternative masking strategies such as

Random, Highest, and Lowest Magnitude outperformed the dense FedAvg baseline, thereby confirming that sparsity itself acts as an effective regularizer against local overfitting, they ultimately fall short of the optimal results. These techniques plateau at lower accuracy levels and exhibit sporadic instability, a behavior we attribute to the heuristic and “blind” nature of their masks. Unlike the sensitivity-based approach, these methods inevitably allow modifications to sensitive parameters, partially degrading the pre-trained representation. Finally, the Most-Sensitive strategy proved to be detrimental, resulting in the most unstable training dynamics. This failure empirically validates the

theoretical hypothesis that modifying weights maximizing Fisher Information triggers catastrophic interference with the features learned by the DINO model.

Contrary to expectations, our experiments did not reveal a definite correlation between increasing calibration rounds and final performance. However, we attribute the lack of a clear trend to the stochastic nature of client selection in Federated Learning, rather than to the ineffectiveness of calibration itself. It is likely that the variance introduced by the random sampling of participants at each round masks the benefits derived from a more precise Fisher matrix estimation. Consequently, to effectively decouple the impact of calibration from process noise and draw statistically robust conclusions, multiple independent executions averaged runs would be required to mitigate the randomness inherent in federated training dynamics.

In conclusion, while sparsity alone offers computational and regularization benefits over FedAvg, it is the sensitivity metric that makes the qualitative difference. Task Arithmetic does not merely reduce the number of parameters to train: it actively protects the model structure, confirming itself as the optimal strategy for efficient federated adaptation.

References

- [1] Mathilde Caron, Hugo Touvron, Ishan Misra, Hervé Jégou, Julien Mairal, Piotr Bojanowski, and Armand Joulin. Emerging properties in self-supervised vision transformers. In *Proceedings of the IEEE/CVF international conference on computer vision*, pages 9650–9660, 2021. [1](#), [2](#)
- [2] Gabriel Ilharco, Marco Tulio Ribeiro, Mitchell Wortsman, Suchin Gururangan, Ludwig Schmidt, Hannaneh Hajishirzi, and Ali Farhadi. Editing models with task arithmetic. In *International Conference on Learning Representations (ICLR)*, 2023. [2](#), [3](#)
- [3] Leonardo Iurada, Marco Ciccone, and Tatiana Tommasi. Efficient model editing with task-localized sparse fine-tuning. In *International Conference on Learning Representations (ICLR)*, 2025. [2](#), [3](#)
- [4] Peter Kairouz, H Brendan McMahan, et al. Advances and open problems in federated learning. *Foundations and Trends® in Machine Learning*, 14(1–2):1–210, 2021. [1](#), [2](#)
- [5] Sylvestre-Alvise Rebuffi, Alexander Kolesnikov, Georg Sperl, and Christoph H Lampert. icarl: Incremental classifier and representation learning. In *Proceedings of the IEEE conference on Computer Vision and Pattern Recognition*, pages 2001–2010, 2017. [1](#), [2](#)